

個人行事曆與會議廳預約系統 — 設計規格

- 日期：2026-08-29
- 狀態：已核可，待撰寫實作計劃

1. 目標與定位

一套**單一公司內部使用**的網頁系統，同時解決兩件事：

1. 員工管理自己的日程（個人行事曆）
2. 員工預約公司的會議廳，且系統保證同一間會議廳在同一時段不會被重複預約

兩者共用同一個事件模型 —— 一筆「會議」就是一個事件，可以掛上會議室與與會者。使用者不需要在兩個系統之間切換。

規模假設：數十至數百名員工、十幾間會議廳、單一組織、無多租戶需求。

2. 範圍

2.1 本規格涵蓋

- 帳號與登入（系統內建帳號密碼）
- 個人行事曆：月／週／日檢視、事件建立與編輯
- 重複事件（完整 RFC 5545 RRULE 語法，有界展開）
- 會議廳主檔維護與空房查詢
- 會議廳預約與衝突偵測（先到先得、即時生效）
- 與會者名單（無回覆流程）
- 站內通知中心
- .ics 單筆匯出與個人訂閱 feed
- 管理員後台：會議廳維護、員工帳號維護

2.2 明確不做（本階段）

不做的項目	理由
多租戶	單一公司內部工具，不需要租戶隔離
審批流程	採先到先得、即時生效

不做的項目	理由
與會者接受／婉拒（RSVP）	名單已解決主要需求，日後可加而不動架構
與 Outlook／Google 雙向同步	屬獨立子專案；本階段僅單向 .ics 輸出
「此筆及後續」的系列編輯	語意複雜且是缺陷溫床，僅提供「這一筆」與「整個系列」
Email 通知、會前提醒推播	需 SMTP 與常駐排程器，站內通知已覆蓋核心場景
無結束日的重複事件	見 5.3；會議廳不應被無限期占用
AD／LDAP／SSO 整合	系統自包含，不依賴公司既有基礎設施

3. 角色與權限

先到先得模式下不需要審批者，因此**只有兩種角色**：

角色	權限
Employee（一般員工）	管理自己建立的事件；預約任何啟用中的會議廳；查看所有會議廳的占用情形；查看自己被邀請的事件
Admin（系統管理員）	員工的全部權限，外加：會議廳主檔的新增／編輯／停用、員工帳號的建立／停用／重設密碼、 強制取消任何人的預約 （用於臨時徵用會議廳）

可見性規則：所有員工都能看到任一會議廳「哪些時段被占用、由誰預約、事件標題」——資源排程本來就需要透明。但**未掛會議廳的純個人事件僅擁有者與其與會者可見**。

4. 資料模型

全系統時間一律以 **Asia/Taipei** 當地時間存放於 `datetime2`，不做 UTC 轉換。台灣無日光節約時間，引入 UTC 只會增加複雜度而無收益。`.ics` 輸出時標註 `TZID:Asia/Taipei`。

4.1 Department

欄位	型別	說明
Id	int, PK	
Name	nvarchar(50), NOT NULL	部門名稱，唯一
IsActive	bit	

4.2 User

欄位	型別	說明
Id	int, PK	
EmployeeNo	nvarchar(20), NOT NULL	員工編號，唯一索引
DisplayName	nvarchar(50), NOT NULL	
Email	nvarchar(100), NOT NULL	唯一索引
DepartmentId	int, FK → Department	
PasswordHash	nvarchar(200), NOT NULL	以 PasswordHasher<User> 產生，見 6.2
Role	nvarchar(20), NOT NULL	Employee / Admin
IcsFeedToken	nvarchar(64), NOT NULL	隨機值，訂閱 feed 的授權憑證，唯一索引
IsActive	bit	停用帳號不能登入，既有事件保留

4.3 Room（會議廳）

欄位	型別	說明
Id	int, PK	
Name	nvarchar(50), NOT NULL	如「A 棟 3F 大會議廳」，唯一
Location	nvarchar(100)	樓層／位置描述
Capacity	int, NOT NULL	容納人數，供空房查詢過濾
Equipment	nvarchar(200)	設備描述（投影機、視訊設備...），純文字
IsActive	bit	停用後不可新增預約，既有預約保留

本表不設 rowversion 樂觀併發欄位 —— 併發控制採 5.2 的悲觀鎖，兩者混用只會製造混淆。

4.4 Event（事件本體 / 系列定義）

欄位	型別	說明
Id	int, PK	
Title	nvarchar(100), NOT NULL	
Description	nvarchar(1000)	
OwnerId	int, FK → User , NOT NULL	建立者

欄位	型別	說明
RoomId	int?, FK → Room	NULL 表示純個人事件，不占用資源
StartAt	datetime2, NOT NULL	系列首次發生的起始時間
EndAt	datetime2, NOT NULL	系列首次發生的結束時間
IsAllDay	bit	全天事件，時間部分固定為 00:00–23:59
RecurrenceRule	nvarchar(500)	RRULE 字串，NULL 表示單次事件
Status	nvarchar(20)	Active / Cancelled
CreatedAt / UpdatedAt	datetime2	

4.5 EventOccurrence（唯一的權威占用表）

設計要點：單次事件也一律產生 1 筆 **occurrence**。系統中不存在「單次事件」與「重複事件」兩條查詢路徑——行事曆顯示與衝突偵測永遠只讀這一張表。

欄位	型別	說明
Id	int, PK	
EventId	int, FK → Event , NOT NULL	
OriginalStartAt	datetime2, NOT NULL	展開時的原始起始時間，等同 iCalendar 的 RECURRENCE-ID， 用來在系列重新展開時辨識「這是哪一次發生」
StartAt	datetime2, NOT NULL	實際起始（單筆編輯後可能不同於 OriginalStartAt）
EndAt	datetime2, NOT NULL	實際結束
RoomId	int?, FK → Room	展開時自 Event 複製；衝突偵測直接查此欄
TitleOverride	nvarchar(100)	單筆編輯時覆寫標題，NULL 表示沿用系列標題
IsModified	bit	該次發生已被單獨修改
IsCancelled	bit	該次發生已被單獨取消

索引：

- 唯一索引 (EventId, OriginalStartAt) —— 一次發生只能有一列
- 篩選索引 (RoomId, StartAt, EndAt) WHERE IsCancelled = 0 AND RoomId IS NOT NULL —— 衝突偵測的主要查詢路徑
- 索引 (StartAt, EndAt) —— 行事曆區間查詢

4.6 EventAttendee

與會者掛在系列層級，不做逐次發生的差異化名單。

欄位	型別	說明
EventId	int, FK, PK	複合主鍵
UserId	int, FK → User , PK	

4.7 Notification

欄位	型別	說明
Id	int, PK	
UserId	int, FK → User , NOT NULL	收件者
Type	nvarchar(30)	AddedToEvent / EventUpdated / EventCancelled / ForcedCancellation
EventId	int?, FK → Event	事件被硬刪除時設為 NULL
Message	nvarchar(300), NOT NULL	產生當下就寫成完整句子，避免日後渲染依賴已變動的事件資料
IsRead	bit	
CreatedAt	datetime2	

索引 (UserId, IsRead, CreatedAt DESC)。

5. 核心業務規則

5.1 衝突規則 —— 只有一條硬性約束

情境	行為
同一會議廳時段重疊	硬性阻擋，回 HTTP 409 並附衝突明細
未指派會議廳的個人事件互相重疊	完全允許，不做任何檢查
與會者本人在該時段已有其他行程	僅警示不阻擋，前端提示「王小明該時段已有 2 場會議」，使用者可逕行送出

重疊判定： 新起 < 舊迄 AND 新迄 > 舊起。**頭尾相接不算衝突** —— 09:00–10:00 與 10:00–11:00 可並存。

衝突檢查的比對範圍：目標 RoomId 下所有 IsCancelled = 0 的 occurrence，且編輯既有事件時**排除該事件自己的 occurrence**。

5.2 防止併發雙重預約

這是整個系統最核心的正確性保證。時間區間重疊無法用唯一索引表達，因此靠交易加鎖：

```
BEGIN TRANSACTION
1. 以 UPDLOCK, HOLDLOCK 讀取目標 Room 資料列
   → 把「同一間會議廳」的所有預約寫入序列化
2. 展開 occurrence 清單（記憶體運算，不落庫）
3. 查詢該 Room 既有 occurrence，逐筆比對重疊
4. 有衝突 → ROLLBACK，回 409
5. 無衝突 → 寫入 Event、EventOccurrence、EventAttendee、Notification
COMMIT
```

未指派會議廳的個人事件不需要取鎖，也不做步驟 3。

鎖的粒度是「單一會議廳」：不同會議廳的預約可以完全並行，不會互相阻塞。

5.3 重複事件

語法完整、範圍有界。支援完整 RRULE 語法（FREQ 全部五種、INTERVAL、BYDAY、BYMONTHDAY、BYSETPOS、BYMONTH），由 Ical.Net 負責解析與展開，但受兩條約束：

1. **必須有結束條件** —— UNTIL 或 COUNT 擇一，拒絕無限期規則
2. **展開上限 730 筆** —— 超過即拒絕，回 400 並提示「重複次數超過上限，請縮短結束日期」

理由：會議廳不應被單一預約無限期占用；有界展開讓衝突偵測成為一句 SQL，正確性可測、可證。

使用者永遠不會看到或輸入 RRULE 字串。畫面提供結構化的重複設定器：

- 頻率下拉：不重複／每天／每週／每月／每年
- 間隔數字：每 N 天／週／月／年
- 每週時：星期核取方塊（可複選）
- 每月時：二選一 —— 「每月 N 日」或「每月第 N 個星期 X」
- 結束條件：指定結束日期，或指定重複次數（**必填**）

後端 RecurrenceService 負責結構化設定 ↔ RRULE 字串的雙向轉換。

5.4 系列編輯語意

僅兩種模式，由 API 的 mode 參數指定：

模式	動作	行為
single	編輯	更新該 occurrence 的 StartAt / EndAt / TitleOverride，設 IsModified = true。 不改動 Event
single	取消	設該 occurrence IsCancelled = true，該時段釋出
series	編輯	更新 Event，然後重新展開： 刪除 IsModified = false AND IsCancelled = false 的 occurrence，依新規則重新產生；OriginalStartAt 已存在於保留列的日期則跳過
series	取消	Event.Status = Cancelled，所有 occurrence 設 IsCancelled = true

mode=single 不可變更會議廳。 要換會議廳只能取消該次發生後另建事件。單筆編輯僅能改時間與標題。

所有會改動時段的編輯，一律走 5.2 的交易與鎖流程—— 包含 mode=single 的時間調整。把一次發生移到別的時段同樣可能撞上既有預約，若這條路徑用單純的 UPDATE 實作，系統最核心的正確性保證就從側門被繞過。僅改標題（TitleOverride）不涉及時段，可略過鎖與檢查。

衝突時整個編輯失敗並回 409，不做部分套用。

關鍵性質：已被單獨修改或取消的 occurrence，在系列重新展開時必須保留。 這是 OriginalStartAt 欄位存在的唯一理由，也是必測項目。

重新展開只作用於當下時間之後的 occurrence。 已經發生過的次數一律原狀保留，即使系列規則改變也不回頭改寫歷史——行事曆的過去紀錄應該反映實際發生過的事。

5.5 通知產生時機

動作	通知對象	類型
建立事件並指定與會者	全體與會者（不含擁有者）	AddedToEvent
編輯事件的時間或會議廳	全體與會者	EventUpdated
取消事件	全體與會者	EventCancelled
管理員強制取消他人預約	事件擁有者 + 全體與會者	ForcedCancellation

mode=single 的編輯與取消同樣發送通知（訊息中標明是哪一次發生，例如「9/14 的『週一產品例會』已改期至 14:00」）——與會者只被改動一次會議，仍然需要知道。

僅修改標題或說明不產生通知。通知訊息在產生當下就寫成完整句子存入 Message。

5.6 .ics 輸出

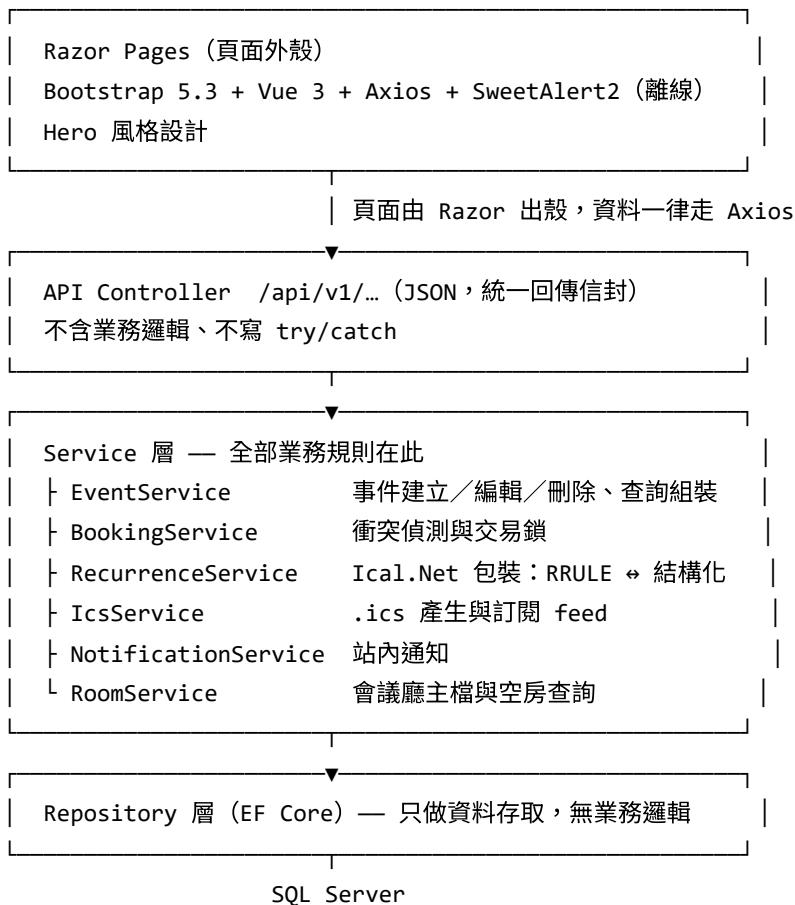
單筆匯出： GET /api/v1/events/{id}/ics 下載該事件的 .ics。

個人訂閱 feed： GET /feeds/{token}.ics

- 匿名端點，以 token 授權，不使用 Cookie —— 行事曆軟體訂閱時無法攜帶登入狀態
- token 為每位使用者的隨機值，個人設定頁可**重新產生以撤銷舊連結**
- 內容 = 該使用者擁有的 + 被邀請參加的、IsCancelled = false 的 occurrence
- **輸出已展開的逐筆 VEVENT，不輸出 RRULE** —— 相容性最好，且與資料庫的權威占用表完全一致
- 每筆 VEVENT 的 UID 使用 {occurrenceId}@calendar.local，DTSTART / DTEND 帶 TZID=Asia/Taipei

6. 系統架構

依 net-core-app 技能的 N-Layer 規範：



6.1 邊界原則

- **BookingService 是唯一能寫入 EventOccurrence 的地方。** 任何繞過它的寫入路徑都可能造成雙重預約，因此這條路徑收斂成單一入口。EventService 需要建立或重新展開 occurrence 時，一律呼叫 BookingService。
- **RecurrenceService 把 Ical.Net 完全包住。** 其他層只認識自訂的 RecurrencePattern DTO，不出現任何 iCalendar 型別。日後若更換函式庫，影響範圍限於此一服務。
- **Repository 不含商業判斷。** 「有沒有衝突」是 Service 的職責；Repository 只提供「取得某會議廳某區間的 occurrence」這類查詢。

6.2 技術棧

項目	選型
執行環境	.NET 10（本機 SDK 10.0.400）
Web 框架	ASP.NET Core（Web API + Razor Pages 同一專案）
ORM	EF Core 10
資料庫	SQL Server；開發與測試使用 SQL Server LocalDB
身分驗證	Cookie 驗證中介軟體 + PasswordHasher<User>（見下方說明）
iCalendar	Ical.Net 5.2.3
前端	Bootstrap 5.3、Vue 3、Axios、SweetAlert2，全部離線放置於 wwwroot
測試	xUnit

不使用 SQLite。 UPDLOCK，HOLDLOCK 是 SQL Server 專屬語法，而它承載本系統最核心的正確性保證——開發與測試環境必須與正式環境使用同一種資料庫，否則併發測試沒有意義。

不引入完整的 ASP.NET Core Identity 框架。 使用 4.2 定義的自訂 User 資料表，搭配 Microsoft.AspNetCore.Identity 套件中的 PasswordHasher<User> 做密碼雜湊，以及 AddAuthentication().AddCookie() 做登入狀態。完整 Identity 會帶進AspNetUsers 等一整組自有結構描述，與本系統的資料模型重複且互相牴觸；本系統只需要「雜湊密碼」與「Cookie 登入」這兩項能力。

7. API 契約

所有 /api/v1/* 端點回傳統一信封：

```
{ "success": true, "data": {}, "message": "", "errors": [] }
```

方法與路徑	用途	權限
POST /api/v1/auth/login	登入，發 Cookie	匿名
POST /api/v1/auth/logout	登出	已登入
GET /api/v1/events?from=&to=&scope=me room all	行事曆區間查詢，讀 occurrence。三種 scope 的定義見 7.3	已登入
GET /api/v1/events/{id}	事件明細 （含重複設定與與會者）	擁有者、與會者，或 （事件已掛會議廳時）

方法與路徑	用途	權限
		任何已登入者
POST /api/v1/events	建立事件	已登入
PUT /api/v1/events/{id}?mode=single series	編輯。 mode=single 需附 occurrenceId	擁有者／Admin
DELETE /api/v1/events/{id}?mode=single series	取消。 mode=single 需附 occurrenceId	擁有者／Admin
GET /api/v1/events/{id}/ics	單筆 .ics 下載	同事件明細
POST /api/v1/events/check-attendees	送出前查詢與會者行程衝突（僅警示用）	已登入
GET /api/v1/rooms	會議廳清單	已登入
GET /api/v1/rooms/availability?date=&capacity=	指定日期各會議廳的占用時段，可依人數過濾	已登入
POST /api/v1/rooms 、 PUT /api/v1/rooms/{id}	會議廳維護	Admin
GET /api/v1/notifications?unreadOnly=	通知清單	已登入
POST /api/v1/notifications/{id}/read	標記已讀	收件者本人
POST /api/v1/users 、 PUT /api/v1/users/{id}	員工帳號維護	Admin
POST /api/v1/me/reset-feed-token	重新產生訂閱 token	已登入
GET /feeds/{token}.ics	個人訂閱 feed	匿名，token 授權

7.1 建立事件的請求範例

```
{
  "title": "週一產品例會",
  "description": "",
  "roomId": 3,
  "startAt": "2026-09-07T10:00:00",
  "endAt": "2026-09-07T11:00:00",
  "isAllDay": false,
  "attendeeIds": [12, 15, 23],
  "recurrence": {
    "frequency": "Weekly",
    "interval": 1,
    "byWeekDays": ["Monday"],
    "endMode": "UntilDate",
    "untilDate": "2026-12-28"
  }
}
```

`recurrence` 為 `null` 表示單次事件。

7.2 衝突回應 (HTTP 409)

```
{
  "success": false,
  "message": "會議廳於下列時段已被預約",
  "errors": [],
  "data": {
    "conflicts": [
      {
        "occurrenceId": 881,
        "roomName": "A 棟 3F 大會議廳",
        "startAt": "2026-09-14T10:00:00",
        "endAt": "2026-09-14T11:00:00",
        "ownerName": "陳大明",
        "title": "季度檢討會"
      }
    ]
  }
}
```

前端以 SweetAlert2 呈現。重複事件若有多次發生衝突，`conflicts` 會列出全部 —— 整筆預約失敗，不做部分套用。

7.3 scope 參數的三種值

值	回傳內容	用於
me	目前登入者擁有的 + 被邀請參加的 occurrence	我的行事曆頁
room	指定 roomId 的所有 occurrence（不分擁有者）	單一會議廳的占用檢視
all	所有已掛會議廳的 occurrence（不含他人的純個人事件）	會議廳資源時間軸頁

scope=room 未附 roomId 時回 400。三種 scope 皆排除 IsCancelled = true 的 occurrence。

7.4 與會者衝突警示的判定

POST /api/v1/events/check-attendees 接收與會者 ID 清單與時段清單，對每位與會者查詢其擁有或被邀請的 occurrence 是否與任一時段重疊（判定規則同 5.1，頭尾相接不算），回傳每人的衝突次數與事件標題。

此端點純屬提示，不影響建立事件的成敗——前端未呼叫它、或使用者忽略警示逕行送出，POST /api/v1/events 都會照常受理。

8. 畫面規格

頁面	內容
登入頁	員工編號 + 密碼
我的行事曆	月／週／日三種檢視切換；顯示自己擁有與被邀請的事件；點空白格快速建立
會議廳資源時間軸	橫軸為時間、縱軸為各會議廳 的當日甘特式檢視，一眼看出空檔；點空白區塊直接帶入該會議廳與時段開始預約
事件建立／編輯彈窗	標題、說明、時間、會議廳下拉（含容納人數）、與會者多選、結構化重複設定器；送出前即時顯示與會者行程衝突警示
事件明細彈窗	完整資訊、與會者名單、「這一筆／整個系列」的編輯與取消按鈕、單筆 .ics 下載
通知中心	導覽列未讀紅點；下拉清單；點擊跳至對應事件
個人設定	修改密碼、訂閱 feed 網址顯示與複製、重新產生 token
會議廳管理 （Admin）	會議廳 CRUD、停用
員工管理（Admin）	帳號 CRUD、指派部門與角色、重設密碼、停用

前端資源全部離線放置於 wwwroot，不引用任何 CDN。

9. 錯誤處理

- Service 層拋領域例外
(`ConflictException` 、 `NotFoundException` 、 `ValidationException` 、 `ForbiddenException`)
- 全域例外處理 `middleware` 統一轉為對應 HTTP 狀態碼與回傳信封；**Controller 內不寫 `try/catch`**
- 對應關係： `ConflictException` → 409、`NotFoundException` → 404、`ValidationException` → 400、`ForbiddenException` → 403、未預期例外 → 500（記錄完整堆疊，回應僅給通用訊息）
- 前端 Axios 攔截器統一處理：409 顯示衝突明細、401 導向登入頁、其餘以 `SweetAlert2` 顯示 `message`

10. 測試策略

依 TDD：每項功能先寫失敗的測試再實作。

10.1 RecurrenceService 單元測試

- 結構化設定 ↔ `RRULE` 字串雙向轉換（每週一、每月最後一個週五、每兩週的週二與週四、每年）
- 展開結果的邊界：跨月、跨年、`COUNT` 剛好用盡
- 無結束條件的規則必須被拒絕
- 展開超過 730 筆必須被拒絕

10.2 BookingService 單元測試

- 重疊判定：完全重疊、部分重疊、包含、被包含 → 全部視為衝突
- 頭尾相接（09:00–10:00 vs 10:00–11:00）→ 不算衝突
- 已取消的 `occurrence` 不參與衝突判定
- 編輯既有事件時，排除該事件自己的 `occurrence`
- 未指派會議廳的事件不做衝突檢查
- 系列重新展開時，`IsModified` 與 `IsCancelled` 的 `occurrence` 必須保留
- 系列重新展開不改動已發生過的 `occurrence`
- `mode=single` 把一次發生移到已被占用的時段 → 必須回 409（驗證單筆編輯沒有繞過衝突檢查）
- `mode=single` 僅改標題 → 不觸發衝突檢查，直接成功
- 重複事件中任一次發生衝突 → 整筆失敗，資料庫無任何寫入

10.3 併發整合測試（驗收核心）

以 SQL Server LocalDB 執行：兩個執行緒同時對同一會議廳、同一時段送出預約，斷言恰好一個成功、另一個收到 409，且資料庫中該時段只有一筆 `occurrence`。此測試重複執行 50 輪以確保穩定。

再加一項：兩個執行緒同時預約不同會議廳的相同時段 → 兩者都應成功，驗證鎖的粒度正確。

10.4 API 整合測試

涵蓋各端點的權限判定（一般員工不能改他人事件、非 Admin 不能維護會議廳、feed token 錯誤回 404）與 .ics 輸出格式（可被行事曆軟體解析、TZID 正確、已取消的 occurrence 不出現）。

11. 驗收標準

1. 員工可登入、建立個人事件、在月／週／日三種檢視中看到它
2. 員工可預約會議廳；重複預約同一會議廳的重疊時段會被拒絕並看到衝突明細
3. 可建立完整 RRULE 語法的重複事件（含「每月第二個星期三」這類規則），且可單獨修改或取消其中一次而不影響其他次
4. 修改整個系列後，先前被單獨修改／取消的那幾次仍維持原樣
5. 併發測試連續 50 輪皆為「恰好一個成功」
6. 與會者收到站內通知；事件改期或取消時再次收到通知
7. 訂閱 feed 網址可被 Outlook 訂閱並正確顯示時間；重新產生 token 後舊網址失效
8. 管理員可維護會議廳與員工帳號，並可強制取消他人預約
9. 全部單元測試與整合測試通過